

○ دستورات `beq, sw, lw` هم از دستورات پایه هستند که از قبل هم در مسیر داده وجود داشتند.

2. اضافه کردن دستورات جدید:

- I-Type: xori

- برای دستور بالا مجبور هستیم که یک تابع جدید به ALU اضافه کنیم چون با توابع کنونی آن برای پیاد سازی تابع xor نیازمند بیش از یک کلاک هستیم و این با نحوه پیاد سازی ما (single-cycle) در تناقض است. پس تابع xor را به ALU اضافه می کنیم.

- I-Type: jalr

- تفاوت این دستور با دستور jal به تنها در نحوه مقداردهی pc است:

$$\text{jal rd, imm_20bit} \longrightarrow \begin{cases} \text{rd} \leq \text{pc} + 4 \\ \text{pc} \leq \text{pc} + \text{imm_20bit} \end{cases}$$

$$\text{jal rd, rs, imm_12bit} \longrightarrow \begin{cases} \text{rd} \leq \text{pc} + 4 \\ \text{pc} \leq \text{rs} + \text{imm_12bit} \end{cases}$$

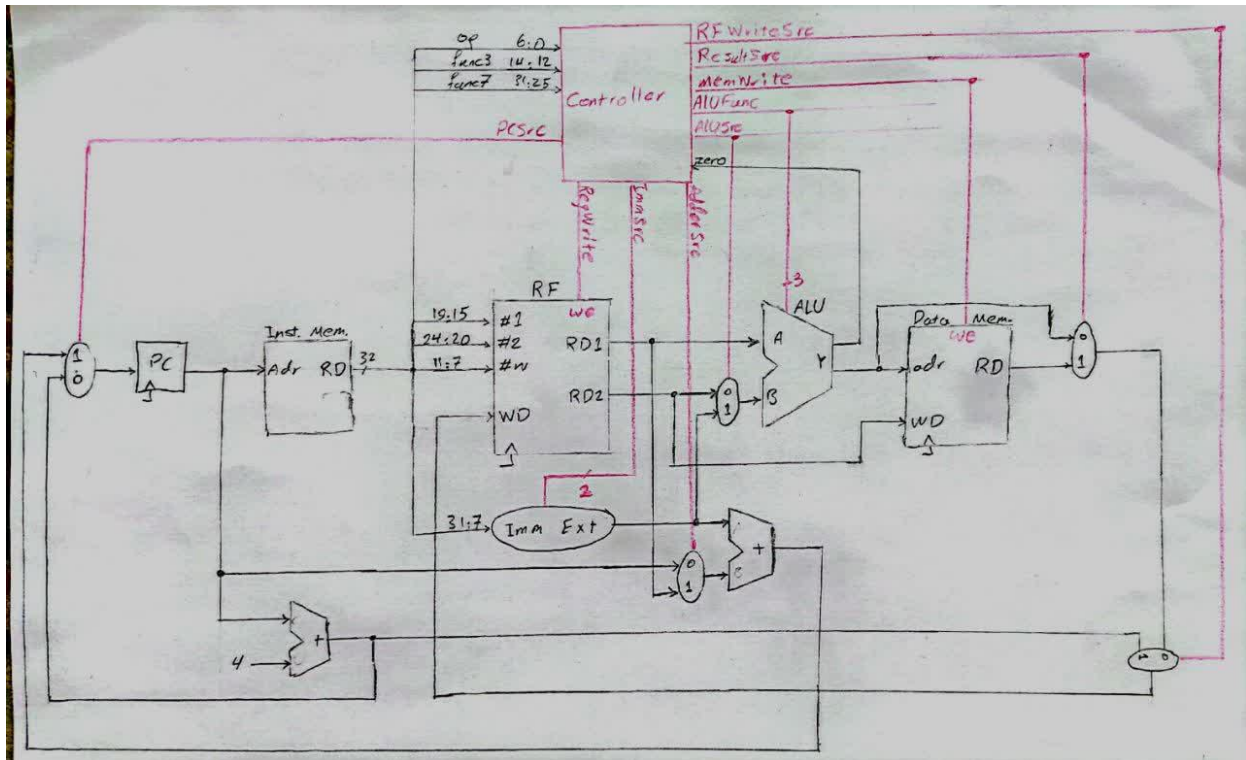
- در نتیجه برای پیاد سازی آن کافی است در مسیر محاسبه مقدار جدید برای pc که برای jal فراهم شده است یک مولتی پلکسر قرار بدهیم تا از بین خروجی رجیسترفایل (rs) و مقدار فعلی pc یکی را انتخاب و با خروجی Imm Ext جمع کند.

- B-Type: bne, blt, bge

- با توجه به اینکه از قبل دستور beq را داریم پیاده سازی آنها خیلی شبیه به آن و ساده خواهد بود:
- bne: کافی است برعکس شرط beq را بررسی کنیم یعنی در صورتی پرش را انجام دهیم که اختلاف دو عدد صفر نباشد و در نتیجه سیگنال zero برابر صفر باشد.
- blt: برای آن از تابع ALU slt استفاده میکنیم به این صورت اگر ورودی A کوچکتر از B باشد خروجی ALU یعنی Y برابر 1 می شود و در نتیجه سیگنال zero صفر می شود. پس میتوانیم براساس zero تصمیم گیری کنیم که اگر صفر بود پرش انجام شود.
- Bge: برعکس دستور قبل عمل میکنیم چون در این دستور اگر ورودی A بزرگتر یا مساوی B باشد باید پرش را انجام دهیم و اگر این اتفاق بیفتد خروجی Y در حالت slt برابر 0 می شود که باز هم میتوانیم از سیگنال zero استفاده کنیم که اگر یک باشد پرش را انجام دهیم.

3. کشیدن مسیر داده:

- طبق توضیحات بخش قبل تنها باید یک مولتی پلکسر به مسیر داده اضافه کنیم. بر روی ورودی دوم adder بالاتر که ورودی های آن pc و RD1 است.



طراحی کنترلر:

1. ALUFunc:

ALU Func	Y
000	$A+B$ (add)
001	$A-B$ (sub)
010	$A \& B$ (and)
011	$A B$ (or)
100	$A < B$ (set less than)
101	$A \oplus B$ (xor)

- طبق توضیحاتی که قبل تر داده شد یک تابع جدید XOR به حالت های قبلی اضافه می کنیم:

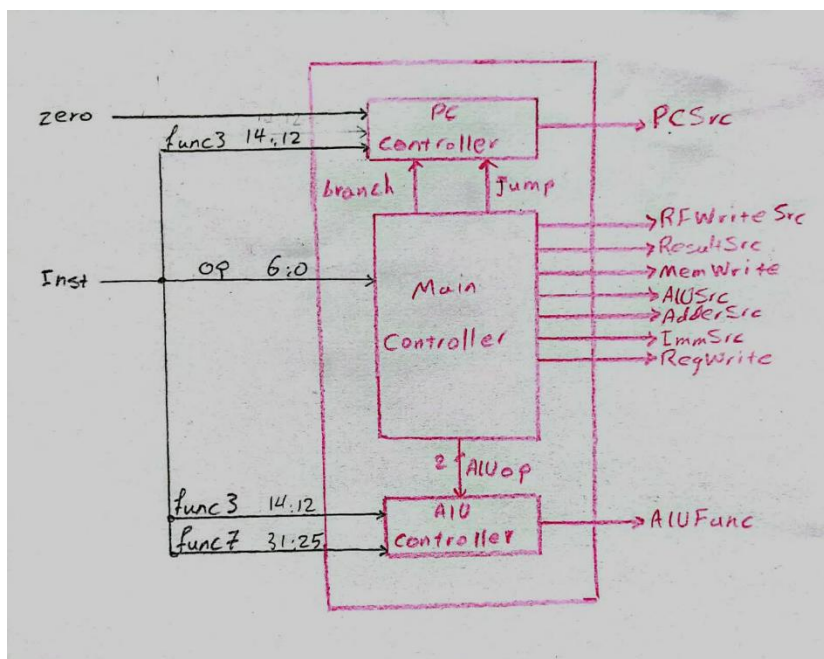
2. ImmSrc:

- با توجه اینکه تایپ جدیدی از دستورات را اضافه نکرده ایم میتوانیم از همان حالت مثال جزوه برای استخراج مقدار فوری استفاده کنیم:

Imm Src	Imm Ext	Inst Type
00	$\{ \{ 20 \{ Inst[31] \} \}, Inst[31:20] \}$	I-Type (lw)
01	$\{ \{ 20 \{ Inst[31] \} \}, Inst[31:25], Inst[11:7] \}$	S-Type (sw)
10	$\{ \{ 19 \{ Inst[31] \} \}, Inst[31], Inst[7], Inst[30:25], Inst[11:8], 1'b0 \}$	B-Type (bge)
11	$\{ \{ 12 \{ Inst[31] \} \}, Inst[19:12], Inst[20], Inst[30:21], 1'b0 \}$	J-Type (jal)

3. شکل کلی کنترلر:

- با توجه به اضافه کردن تعداد زیاد دستور که باعث تغییر مقدار pc و عمل پرش (bne, blt, bge, jalr) می شوند در نتیجه باید برای تشخیص سیگنال مولتی پلکسر ورودی آن یک واحد مجزا در نظر بگیریم و سیگنال jump را برای دو دستور jal و jalr اضافه کنیم.



- همچنین سیگنال دیگری برای برای مولتی پلکسری که به مسیر داده اضافه شد باید در نظر بگیریم.

4. جزئیات سیگنال های کنترل:

	op	op code	func3	func7	RFWrite Src	Result Src	Mem Write	ALU Src	Adder Src	Imm Src	Reg Write	Branch	Jump	ALUop	PCSrc	ALUFunc
R_Type	add	0110011	000	0000000	0	0	0	0	X	XX	1	0	0	10	0	000
	sub		000	0100000	0	0	0	0	X	XX	1	0	0	10	0	001
	and		111	0000000	0	0	0	0	X	XX	1	0	0	10	0	010
	or		110	0000000	0	0	0	0	X	XX	1	0	0	10	0	011
I_Type	slt		010	0000000	0	0	0	0	X	XX	1	0	0	10	0	100
	lw	0000011	010	-	0	1	0	1	X	00	1	0	0	00	0	000
	addi	0010011	000		0	0	0	1	X	00	1	0	0	11	0	000
	xori		100		0	0	0	1	X	00	1	0	0	11	0	101
	ori		110		0	0	0	1	X	00	1	0	0	11	0	011
	sli		010		0	0	0	1	X	00	1	0	0	11	0	100
	jalr	1100111	000		1	X	0	X	1	00	1	0	1	XX	0	XXX
	sw	0100011	010	-	X	X	1	1	X	01	0	0	0	00	0	000
S_Type	jal	1101111	-	-	1	X	0	X	0	11	1	0	1	XX	0	XXX
B_Type	beq	1100011	000	-	X	X	0	0	0	10	0	1	0	01	zero	001
	bne		001		X	X	0	0	0	10	0	1	0	01	~zero	001
	blt		100		X	X	0	0	0	10	0	1	0	01	~zero	100
	bge		101		X	X	0	0	0	10	0	1	0	01	zero	100

پیاده سازی مسیر داده:

1. برای پیاده سازی مسیر داده، ابتدا تمام ماژول هایی که در آن آمده است را طراحی می کنیم. حافظه ها با این که در این بخش قرار ندارند و خارج پردازنده به آن نصب می شوند، ولی باز هم برای متصل کردن اتصالات درست آن ها را در این بخش طراحی می کنیم و فقط متصل کردنشان را به آخر کار موکول می کنیم:

- ALU
- Multiplexer2to1
- PC
- Sum
- RF
- Inst_mem
- Data_Mem
- Imm_Ext

در این جا طراحی می کنیم ولی در بیرون پردازنده آن را نصب می کنیم

2. حال نوبت پیاده سازی مسیر داده است. کفایت ماژول های طراحی شده را (به جز حافظه ها را که در انتها با آن ها سر و کار داریم) به یک دیگر به صورت ساختاری متصل کنیم. اتصالات به کنترل و واحد های حافظه را به همراه کلاک نیز در این جا به صورت ورودی و خروجی در نظر خواهیم گرفت:

```
module DataPath
(
    input clk, PCSrc, RegWrite, AdderSrc, ALUSrc, ResultSrc, RFWriteSrc,
    input [1:0] ImmSrc,
    input [2:0] ALUFunc,
    input [31:0] inst, MemData,
    output zero,
    output [31:0] PC_out, ALU_out, RegData2
);

    wire [31:0] PC_in, RegWriteData, RegData1, ALU_input2, Imm_out, Sum1_out,
    Sum2_out, Sum2_in, Result_out;

    PC stage0 (clk, PC_in, PC_out);
    RF stage1 (clk, inst[19:15], inst[24:20], inst[11:7], RegWriteData, RegWrite,
    RegData1, RegData2);
    ALU stage2 (RegData1, ALU_input2, ALUFunc, ALU_out, zero);
    Imm_Ext stage3 (inst[31:7], ImmSrc, Imm_out);

    Multiplexer2to1 stage4 (Sum1_out, Sum2_out, PCSrc, PC_in);
    Multiplexer2to1 stage5 (RegData2, Imm_out, ALUSrc, ALU_input2);
    Multiplexer2to1 stage6 (PC_out, RegData1, AdderSrc, Sum2_in);
    Multiplexer2to1 stage7 (ALU_out, MemData, ResultSrc, Result_out);
    Multiplexer2to1 stage8 (Result_out, Sum1_out, RFWriteSrc, RegWriteData);

    Sum stage9 (PC_out, 32'd4, Sum1_out);
    Sum stage10 (Imm_out, Sum2_in, Sum2_out);
endmodule
```

پیاده سازی کنترلر:

1. ما برای پیاده سازی کنترل نیاز است سه ماژول را به صورت جدا گانه پیاده سازی کنیم و با اتصال این سه ماژول به یک دیگر، کنترلر ما به صورت ترکیبی ساخته خواهد شد:

- اولین ماژولی که ما نیاز داریم، MainController است. این ماژول موظف است بر اساس operation code تمام خروجی های کنترلری مورد نیاز را به جز دو خروجی PCSrc و ALU_function فراهم آورد. برای فراهم آوردن این دو نیاز به اطلاعاتی فراتر از operation code است ولی برای تشخیص این دو دستور، باز هم operation code نیاز است. پس این ماژول سه خروجی با نام های branch، jump و ALU_operation برای محاسبه دو خروجی مورد نیاز فراهم می کند.

- دومین ماژول ALUController است. این ماژول موظف است با گرفتن ALU_operation و func3 و func7، نوع دستور را به صورت کامل شناسایی کند و خروجی مناسب را برای ALU بفرستد تا محاسبات به صورت درست انجام شود.
- سومین ماژول PCController است. این ماژول موظف است با گرفتن branch، jump، func3 و همچنین سیگنال ورودی zero، مالتی پلکسر ورودی PC را کنترل کند که آیا پرشی انجام بدهد یا خیر. در اصل سیگنال خروجی PCSrc را به صورت درست مقدار دهی می کند.

2. بعد از طراحی ریز ماژول های کنترلری بالا، نوبت به آن می رسد که این سه ماژول را به هم وصل کنیم و یک کنترلر ترکیبی کامل داشته باشیم:

```
module Controller
(
    input [31:0] inst,
    input zero,
    output PCSrc, RegWrite, AdderSrc, ALUSrc, MemWrite, ResultSrc, RFWriteSrc,
    output [1:0] ImmSrc,
    output [2:0] ALU_function
);

    wire branch, jump;
    wire [1:0] ALU_operation;

    MainController stage0 (inst[6:0], RegWrite, AdderSrc, ALUSrc, MemWrite,
ResultSrc, RFWriteSrc, branch, jump, ImmSrc, ALU_operation);
    ALUController stage1 (inst[14:12], inst[31:25], ALU_operation, ALU_function);
    PCController stage2(inst[14:12], branch, jump, zero, PCSrc);

endmodule
```

پیاده سازی پردازنده RISC-V:

- حال وقت آن می شود که متصل کردن کنترلر و مسیر داده، پردازنده RISC-V را کامل کنیم:

```
module RISCv
(
    input clk,
    input [31:0] inst, MemData,
    output [31:0] PC_out, ALU_out, RegData2,
    output MemWrite
);
    wire PCSrc, RegWrite, AdderSrc, ALUSrc, ResultSrc, RFWriteSrc;
    wire [1:0] ImmSrc;
    wire [2:0] ALUFunc;
    wire zero;

    DataPath stage0(clk, PCSrc, RegWrite, AdderSrc, ALUSrc, ResultSrc, RFWriteSrc,
ImmSrc, ALUFunc, inst, MemData, zero, PC_out, ALU_out, RegData2);
    Controller stage1(inst, zero, PCSrc, RegWrite, AdderSrc, ALUSrc, MemWrite,
ResultSrc, RFWriteSrc, ImmSrc, ALUFunc);

endmodule
```

پیاده سازی TopModule:

- بعد از کامل کردن پردازنده، نوبت به آن رسیده که آن را به واحد های حافظه ی Inst_mem و Data_mem را به پردازنده متصل کرده و Top module، خود را پیاده سازی کنیم که تمامی امکانات را داشته باشد. کد این بخش هم به صورت زیر خواهد بود:

```
module TopModule
(
    input clk
);
    wire [31:0] inst, MemData, PC_out, ALU_out, RegData2;
    wire MemWrite;

    RISCv stage0(clk, inst, MemData, PC_out, ALU_out, RegData2, MemWrite);
    inst_mem stage1 (PC_out, inst);
    Data_Mem stage2 (clk, ALU_out, RegData2, MemWrite, MemData);

endmodule
```

- برای اجرای این ماژول فقط کافیسیت یک تست بنچ بنویسیم که clk را جابجا کند و باقی کار ها را خودش به صورت کامل انجام خواهد داد.

طراحی تست:

- برنامه ای که یک آرایه 10 عنصری از اعداد صحیح علامت دار 32 بیتی را مرتب می کند.

1. انتخاب الگوریتم مرتب سازی:

○ الگوریتم مرتب سازی انتخابی (selection sort) را برای این تست انتخاب می کنیم و علت این انتخاب هم به شرح زیر است:

(1) بهینه سازی و کاهش چشمگیر مراجعات به حافظه (Memory Writes): یکی از مهمترین ویژگی های این الگوریتم مرتب سازی در این است که تعداد مراجعه به حافظه برای نوشتن (استفاده از دستور SW) بسیار کم است زیرا حداکثر $n - 1$ بار باید جابه جایی را انجام دهیم یعنی حداکثر $2n - 2$ بار از SW استفاده میکنیم (اگر تعداد عناصر را n فرض کنیم).

(2) سازگاری کامل با محدودیت های مجموعه دستورات (نبود دستورات شیفت و ضرب): با توجه به عدم وجود دستوراتی مانند slli و mul در این معماری محاسبه آدرس حافظه ($4 * \text{Index}$) کار ساده نیست. در مرتب سازی انتخابی، می توانیم به جای ضرب کردن اندیس ها، مستقیماً یک رجیستر را به عنوان آدرس پایه در نظر گرفته و با دستور `addi rd, rs, 4` به صورت متوالی در طول آرایه حرکت کنیم و آدرس کمترین عنصر را در یک رجیستر جداگانه ذخیره نگه داریم.

(3) سادگی در پیاده سازی کنترل جریان (Control Flow): در این الگوریتم، منطق شرطی بسیار ساده است. در حلقه داخلی، تنها نیاز به یک دستور پرش شرطی (مانند bge برای پرش در صورت بزرگتر یا مساوی بودن) داریم تا در صورت پیدا شدن عدد کوچکتر، مقدار مینیمم جدید ثبت شود. این ساختار سراسر باعث می شود که پیاده سازی حلقه های تو در تو با استفاده از دستورات محدود J-Type (jal) و B-Type بسیار تمیز و با کمترین میزان پرش های تو در تو در اسمبلی انجام شود.

(4) مناسب بودن برای داده های با حجم کم: از آنجا که تعداد عناصر آرایه بسیار کوچک و برابر با $n = 10$ است، پیچیدگی زمانی الگوریتم که $O(n^2)$ است هیچ گونه افت عملکرد محسوسی ایجاد نمی کند. در مقابل، الگوریتم های با پیچیدگی زمانی بهتر (مثل Quick Sort یا Merge Sort) نیازمند مدیریت پیچیده پشته (Stack) و فراخوانی های بازگشتی هستند که با این مجموعه دستورات محدود، پیاده سازی آن ها به شدت سخت و غیرمنطقی است.

○ در نتیجه الگوریتم Selection Sort تعادل کاملی بین سادگی پیاده سازی (با توجه به دستورات محدود R-Type و I-Type) و کارایی (به دلیل حداقل رساندن استفاده از دستور sw) برقرار می کند و برای مرتب سازی 10 عدد صحیح علامت دار در معماری Single-Cycle RISC-V این معماری، بهترین انتخاب محسوب می شود.

2. پیاده سازی کد C++:

```
arr[10] // Assuming the array is given like this
```

```
for (int i = 0; i < n - 1; i++) {  
    int minIndex = i;  
    for (int j = i + 1; j < n; j++) {  
        if (arr[j] < arr[minIndex]) {  
            minIndex = j;  
        }  
    }  
    swap(arr[i], arr[minIndex]);  
}
```

3. تبدیل به دستورات مجاز RISC_V:

○ با استفاده از جدول جزییات سیگنال های کنترلر که func7, func3, op دستورات در آن ذکر شده است دستورات را به فرم زیر تبدیل میکنیم:

1.	addi x8, x0, 10	# x8 = n = 10
2.	addi x9, x0, 9	# x9 = n - 1 = 9
3.	add x5, x0, x0	# x5 = i = 0
4.	add x25, x0, x0	# x25 = i_offset = 0
5.	outer_loop: bge x5, x9, done	# if i >= 9, exit loop
6.	add x11, x0, x5	# minIndex = i
7.	add x26, x0, x25	# minIndex_offset = i_offset
8.	addi x6, x5, 1	# j = i + 1
9.	addi x27, x25, 4	# j_offset = i_offset + 4
10.	inner_loop: bge x6, x8, end_inner	# if j >= 10, exit inner loop
11.	lw x28, 0(x27)	# x28 = arr[j]
12.	lw x29, 0(x26)	# x29 = arr[minIndex]
13.	bge x28, x29, skip	# if arr[j] >= arr[minIndex], skip update
14.	add x11, x0, x6	# minIndex = j
15.	add x26, x0, x27	# minIndex_offset = j_offset
16.	skip: addi x6, x6, 1	# j++
17.	addi x27, x27, 4	# j_offset += 4
18.	jal x0, inner_loop	# jump to inner_loop
19.	end_inner: lw x28, 0(x25)	# x28 = arr[i]
20.	lw x29, 0(x26)	# x29 = arr[minIndex]
21.	sw x29, 0(x25)	# arr[i] = old arr[minIndex]
22.	sw x28, 0(x26)	# arr[minIndex] = old arr[i]
23.	addi x5, x5, 1	# i++
24.	addi x25, x25, 4	# i_offset += 4
25.	jal x0, outer_loop	# jump to outer_loop
26.	done:	

- برای مقادیر پرش با دستورات bge و jal باید دستورات را به صورت 4 حطی فرض کنیم (چون در ادامه هر دستور به 4 خط 8 بیتی تبدیل می شود) و بر اساس آن آدرس خطی که به آن باید پرش کند را پیدا کنیم که به صورت زیر می شود:

- done = +84
- end_inner = +36
- skip = +12
- inner_loop = -32
- outer_loop = -80

○ نتیجه نهایی:

```
1.      addi x8, x0, 10      # x8 = n = 10
2.      addi x9, x0, 9      # x9 = n - 1 = 9
3.      add x5, x0, x0      # x5 = i = 0
4.      add x25, x0, x0     # x25 = i_offset = 0
5.  outer_loop:  bge x5, x9, 84  # if i >= 9, exit loop
6.      add x11, x0, x5      # minIndex = i
7.      add x26, x0, x25     # minIndex_offset = i_offset
8.      addi x6, x5, 1      # j = i + 1
9.      addi x27, x25, 4     # j_offset = i_offset + 4
10. inner_loop:  bge x6, x8, 36  # if j >= 10, exit inner loop
11.     lw x28, 0(x27)        # x28 = arr[j]
12.     lw x29, 0(x26)        # x29 = arr[minIndex]
13.     bge x28, x29, 12      # if arr[j] >= arr[minIndex], skip update
14.     add x11, x0, x6      # minIndex = j
15.     add x26, x0, x27     # minIndex_offset = j_offset
16. skip:      addi x6, x6, 1  # j++
17.     addi x27, x27, 4     # j_offset += 4
18.     jal x0, -32         # jump to inner_loop
19. end inner:  lw x28, 0(x25)  # x28 = arr[i]
20.     lw x29, 0(x26)        # x29 = arr[minIndex]
21.     sw x29, 0(x25)        # arr[i] = old arr[minIndex]
22.     sw x28, 0(x26)        # arr[minIndex] = old arr[i]
23.     addi x5, x5, 1      # i++
24.     addi x25, x25, 4     # i_offset += 4
25.     jal x0, -80         # jump to outer_loop
26. done:
```

4. تبدیل به دستورات باینری براساس op code های RISC_V:

```
1. 0000000010100000000010000010011
2. 00000000100100000000010010010011
3. 00000000000000000000001010110011
4. 000000000000000000000110010110011
5. 00000100100100101101101001100011
6. 00000000010100000000010110110011
7. 00000001100100000000110100110011
8. 00000000000100101000001100010011
9. 00000000010011001000110110010011
10. 00000010100000110101001001100011
11. 00000000000011011010111000000011
12. 00000000000011010010111010000011
13. 00000001110111100101011001100011
14. 00000000011000000000010110110011
15. 00000001101100000000110100110011
16. 00000000000100110000001100010011
17. 00000000010011011000110110010011
18. 11111110000111111111000001101111
19. 00000000000011001010111000000011
20. 00000000000011010010111010000011
21. 00000001110111001010000000100011
22. 00000001110011010010000000100011
23. 00000000000100101000001010010011
24. 00000000010011001000110010010011
25. 11111011000111111111000001101111
```

5. تبدیل به خطوط 8 بیتی:

- در این تبدیل به این صورت عمل میکنیم که بیت های پرارزش تر دستور در خطوط پایین تر آن دستور قرار بگیر که بتوان با استفاده از خط زیر آنها را استخراج کرد:

assign **RD** = {mem[Adr + 3], mem[Adr + 2], mem[Adr + 1], mem[Adr]}

- در نتیجه داریم:

```
1. 00010011
2. 00000100
3. 10100000
4. 00000000
5. 10010011
6. 00000100
7. 10010000
8. 00000000
9. 10110011
10. 00000010
11. 00000000
12. 00000000
13. 10110011
14. 00001100
15. 00000000
16. 00000000
17. 01100011
18. 11011010
19. 10010010
20. 00000100
21. 10110011
22. 00000101
23. 01010000
24. 00000000
25. 00110011
26. 00001101
27. 10010000
28. 00000001
29. 00010011
30. 10000011
31. 00010010
32. 00000000
33. 10010011
34. 10001101
```

```
35. 01001100
36. 00000000
37. 01100011
38. 01010010
39. 10000011
40. 00000010
41. 00000011
42. 10101110
43. 00001101
44. 00000000
45. 10000011
46. 00101110
47. 00001101
48. 00000000
49. 01100011
50. 01010110
51. 11011110
52. 00000001
53. 10110011
54. 00000101
55. 01100000
56. 00000000
57. 00110011
58. 00001101
59. 10110000
60. 00000001
61. 00010011
62. 00000011
63. 00010011
64. 00000000
65. 10010011
66. 10001101
67. 01001101
68. 00000000
```

```
69. 01101111
70. 11110000
71. 00011111
72. 11111110
73. 00000011
74. 10101110
75. 00001100
76. 00000000
77. 10000011
78. 00101110
79. 00001101
80. 00000000
81. 00100011
82. 10100000
83. 11011100
84. 00000001
85. 00100011
86. 00100000
87. 11001101
88. 00000001
89. 10010011
90. 10000010
91. 00010010
92. 00000000
93. 10010011
94. 10001100
95. 01001100
96. 00000000
97. 01101111
98. 11110000
99. 00011111
100. 11111011
```

6. نوشتن آرایه 10 عنصری از اعداد صحیح:

○ برای آرایه از اعداد زیر استفاده میکنیم:

۷۷, ۱-, ۲۳, ۵۰-, ۸, ۵-, ۱۰۰, ۰, ۱۲-, ۴۵ ○

○ که فرم مکمل دوانها به صورت زیر است:

[illegible]

○ مانند قسمت دستورات الگوریتم مرتب سازی در اینجا هم هر عدد را به صورت 32 بیتی در 4 خط می نویسیم که هر خط 8 و خطوط پایین پر ارزش تر هستند:

```

1. 00101101
2. 00000000
3. 00000000
4. 00000000
5. 11110100
6. 11111111
7. 11111111
8. 11111111
9. 00000000
10.00000000
11.00000000
12.00000000
13.01100100
14.00000000
15.00000000
16.00000000
17.11110111
18.11111111

```

19.11111111
20.11111111
21.00001000
22.00000000
23.00000000
24.00000000
25.11001110
26.11111111
27.11111111
28.11111111
29.00010111
30.00000000
31.00000000
32.00000000
33.11111111
34.11111111
35.11111111
36.11111111

```
37. 01001101
38. 00000000
39. 00000000
40. 00000000
```